

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 3 — Esercitazione: una dashboard Artifact da dati sintetici anonimizzati

La stessa dashboard in Claude Code

M3-E03: la stessa dashboard in Claude Code

L'ultima esercitazione del laboratorio, nel vostro Claude Code: dodici minuti

- La settimana di ticket di **CiaspoLink**, anonimizzata in M3-E01: gli stessi **dieci indicatori** della dashboard di M3-E02, l'artifact di claude.ai, ma come **pagina HTML locale**, un file solo che si apre **senza rete**. Claude scrive `genera_dashboard.py`, lo esegue, lancia il **controllo** e corregge finché passa
- Il file lo legge **lo script**, non Claude: Claude Code manda in rete prompt e risposte, e nella conversazione entra ciò che Claude legge e ciò che un comando stampa
- **Percorso A**: un vostro CSV con **dati inventati**, come nel percorso A di M3-E01, con i vostri indicatori e il loro profilo — lo stesso metodo, senza verifica automatica
- **Percorso B**: la settimana di CiaspoLink, i vostri file di M3-E01 o le copie di riferimento — la prova del docente e la verifica sono su questo
- La guida di Claude Code: «**Give Claude a check to run**». Che cosa può essere il controllo: «The check can be almost anything, as long as Claude can read the result: … **A script that diffs output against a fixture**» — qui il riferimento, la *fixture*, è `anonimo/profilo.json`, il profilo scritto da M3-E01. Il ciclo di Claude che mettete alla prova: «It makes the change, runs the check, and if something broke it reads the output and fixes it before running the check again.»
- Il confronto con l'artifact di M3-E02 — chi lo apre, dove sono andati i dati, come si rifà la settimana dopo — è in «Confronto», la parte che chiude il laboratorio

Il prompt e il controllo

Il prompt, il controllo che Claude esegue, le impostazioni dello spazio di lavoro

nella sessione su Sonnet: il prompt del passo 2, 1 di 2

```
Sei uno sviluppatore Python dell'assistenza clienti di
CiaspoLink e prepari la dashboard della settimana come una
pagina che si apre senza rete.

Contesto: la settimana anonima è in anonimo/ticket-anonimi.csv;
i dieci indicatori, con le loro regole, sono in
kpi_definizioni.md.

Istruzione: scrivi ed esegui uno script, in tre passi.
1. Leggi kpi_definizioni.md.
2. Scrivi genera_dashboard.py, che legge
   anonimo/ticket-anonimi.csv, calcola i dieci indicatori con le
   regole di kpi_definizioni.md e scrive dashboard.html, la
   dashboard della settimana. Eseguilo con
   python3 genera_dashboard.py.
```

Il prompt e il controllo

Il prompt, il controllo che Claude esegue, le impostazioni dello spazio di lavoro

nella sessione su Sonnet: il prompt del passo 2, 2 di 2

```
3. Esegui python3 controlla_dashboard.py: confronta i numeri del
   blocco kpi con anonimo/profilo.json e cerca nella pagina ciò
   che arriverebbe da fuori. Correggi genera_dashboard.py, e
   riesegui il generatore e il controllo, finché il controllo
   esce con 0.

Formato: lo script in Python 3.8 o successivo, solo libreria
standard. dashboard.html è un file solo, con CSS e grafici
dentro la pagina, per esempio barre in SVG. I numeri sono
scritti nel testo della pagina da genera_dashboard.py, e tutti e
dieci gli indicatori stanno anche in un blocco
<script type="application/json" id="kpi">, con le chiavi di
kpi_definizioni.md.

Vincoli: anonimo/ticket-anonimi.csv lo legge soltanto lo script,
e il suo contenuto resta fuori da ciò che stampi. Tutto ciò che
la pagina usa, script, fogli di stile, font e immagini, sta
dentro il file. Esegui ogni comando da solo, senza concatenarlo
ad altri, e leggi l'uscita del controllo nella sua ultima riga.
```

Il prompt e il controllo

Il prompt, il controllo che Claude esegue, le impostazioni dello spazio di lavoro

Il controllo della pagina

- `controlla_dashboard.py` confronta il blocco `kpi` della pagina con `anonimo/profilo.json`, il profilo di M3-E01, **chiave per chiave**, con le chiavi che il profilo ha: per ogni indicatore l'atteso, il trovato, ok o NO
- Cerca ciò che la pagina caricherebbe **dalla rete** o **da un altro file** — script, fogli di stile, font, immagini — e uno script della pagina che **chiama la rete**
- L'**ultima riga** è l'esito, «uscita: 0 — …» oppure «uscita: 1 — …»: si legge lì, **un comando alla volta**
- Ogni esecuzione aggiunge una riga a `controlli_dashboard.log`: l'ora, l'uscita, le differenze

Le impostazioni dello spazio di lavoro

- In `.claude/settings.json`, `defaultMode` `default`: la modalità **Manual**, ogni scrittura e ogni altro comando li approvate voi
- `allow` per `python3 genera_dashboard.py` e `python3 controlla_dashboard.py`: **pre-approvati**, non chiedono niente
- `deny` di `Read` sul file anonimo: ferma gli strumenti di Claude per i file e `cat`, `head`, `tail`; non lo script, che lo apre **da sé** — una **guardia**, non un muro
- `deny` di `Edit` sul controllo: si corregge lo script, **non il controllo**

Al lavoro

I passi 1–4 del README: nove minuti, nel vostro Claude Code

```
# dalla cartella esercitazioni: il passo 1 del README
tools/prepara_esercizio.sh M3-E03
cd ~/corso-claude-code/M3-E03
# chi ha fatto M3-E01, con la verifica a 0, copia i suoi file:
cp ../M3-E01/anonimo/{ticket-anonimi.csv,profilo.json} anonimo/
claude --model sonnet --effort medium
# nella sessione: il prompt intero del passo 2 del README
```

- Il README dell'esercitazione: `m3_funzionalita_claude/s2_funzionalita_claude/m3_e4_dashboard_locale/M3-E03_dashboard_locale/`
- **Checkpoint**: `genera_dashboard.py` scritto ed eseguito; il controllo a «**uscita: 0**»; la pagina **nel browser**, dallo spazio di lavoro: `xdg-open dashboard.html` su Linux, con WSL 2 `explorer.exe .` e un doppio clic
- Leggete le richieste: **approvate** la scrittura dello script, le sue correzioni e i comandi che riconoscete (i due `python3` dell'esercizio non chiedono niente); **No** a ogni comando che stamperebbe il file anonimo, come un `python3 -c` che lo apre: la regola deny non lo ferma
- **No** anche a una riga che unisce più comandi, con `&&`, `;` o un `echo $?` in coda: chiedete a Claude **un comando alla volta**, l'esito è l'ultima riga del controllo
- «dalla rete», «da un altro file», un indicatore che non torna: «Se qualcosa va storto», nel README
- L'impronta: una sessione su **Sonnet**, effort **medium**, nessun subagent, sullo stesso limite di claude.ai; controllo e verifica sono script. Nella registrazione (`claude -p`, senza approvazioni): il prompt in **un messaggio**, 6–9 turni e 40–45 secondi a sessione, il controllo **1 volta**

La verifica

Il passo 5 del README: fuori dalla sessione, uno script rifà i conti

```
/exit                       # nella sessione di Claude Code
cd -                        # di nuovo nella cartella esercitazioni
tools/verifica.sh M3-E03    # 0 fatto, 1 non ancora
```

- Ricalcola i dieci indicatori dal file anonimo, con lo script dell'elemento, e legge la pagina con il controllo **originale**, non con le copie dello spazio di lavoro: il blocco `kpi` contro gli indicatori **veri**, e **niente** dalla rete né da altri file
- Stampa, come **dato**: i numeri singoli nel **testo** della pagina; le esecuzioni registrate in `controlli_dashboard.log`, **senza** un esito sul ciclo della guida, perché il registro lo scrive il controllo chiunque lo esegua; se il controllo dello spazio di lavoro è cambiato
- Nella registrazione del corso (31 ottobre 2026, Claude Code 2.1.316, Sonnet, effort medium, `claude -p` senza approvazioni), tre sessioni: in tutte e tre i dieci indicatori **giusti**, **7 su 7** numeri singoli nel testo, niente da fuori; il controllo eseguito una volta in ognuna, con uscita 0; la frase della guida, dalle chiamate di ogni sessione: **confermata 3 su 3**

Lo stesso grafico, due strumenti: un link per chi ha un account Claude, o un file che si rigenera la settimana dopo.